ASSIGNMENT 03 Full Stack

(Priyanshu Sharma B.tech (cse) 2nd Year (B)

2. Functional React Counter Using useState

The useState hook is used to store a value that can change during the lifetime of a React component. Here, count stores the current number and setCount changes it. When the state changes, React automatically re-renders the component and shows the new value.

Answer / Corrected Code:

```
import React, { useState } from "react";

function Counter() {   const [count, setCount] =
useState(0);

 return (    <div>
    <h2>Count: {count}</h2>

    <button onClick={() => setCount(count + 1)}>
      Increment
    </button>

    <button onClick={() => setCount(count - 1)}>
      Decrement
    </button>
  </div>
); }

export default Counter;
```

Explanation: useState(0) creates a state variable named count with an initial value of 0. The setCount function updates that state. Clicking Increment increases the value by 1, while clicking Decrement decreases it by 1. A normal JavaScript variable would not cause React to re-render the component when its value changes, so the updated value would not automatically appear on the screen.

3. Fixing the Infinite Re-render Loop

The problem is that setCount(count + 1) is called directly inside the component's render body. Every time the state changes, React renders the component again. During that new render, setCount() runs again, causing another render, and this continues indefinitely.

Answer / Corrected Code:

```
import React, { useState } from "react";
function Counter() {   const [count, setCount] =
useState(0);

 return (    <div>
    <p>{count}</p>
    <button onClick={() => setCount(count + 1)}>
      Increase
    </button>
  </div>
); }

export default Counter;
```

Explanation: State should not be changed directly during the render process. The update should happen because of an event, such as a button click, or inside an appropriate useEffect with a controlled dependency. In this corrected code, setCount() runs only when the user clicks the button, so the infinite re-render loop is removed.

4. Fetching Data with useEffect on Component Mount

useEffect is used for side effects such as API requests. To fetch posts when the component first appears on the screen, the fetch operation is placed inside useEffect(). The empty dependency array [] tells React that the effect does not depend on changing values and should run after the initial render.

Answer / Corrected Code:

```
import React, { useEffect, useState } from "react";

function Posts() {   const [posts, setPosts] =
useState([]);

  useEffect(() => {    fetch("/api/posts")
  .then(res => res.json())
     .then(data => setPosts(data))
     .catch(error => console.error(error));   }, []);

  return (    <div>
     {posts.map(post => (
       <p key={post.id}>{post.title}</p>
     ))}
   </div>
  ); }

export default Posts;
```

Explanation: The empty dependency array [] is necessary because it makes the effect run after the component's initial render rather than running again after every render. This is suitable for a one-time data fetch when the component first mounts.

5. Lifting Shared State Up to the Parent

The two sibling components each have their own separate price state. Therefore, changing the price in PriceInput does not change the value shown by PriceDisplay. The common state should be moved to their nearest common parent. This React technique is called lifting state up.

Answer / Corrected Code:

```
import React, { useState } from "react";
function PriceInput({ price, setPrice }) {   return
(    <input      value={price}      onChange={e =>
setPrice(e.target.value)}     />
  ); }

function PriceDisplay({ price }) {   return
<p>Price: {price}</p>; }

function ProductPrice() {   const [price, setPrice] =
useState(0);

  return (    <div>
     <PriceInput price={price} setPrice={setPrice} />
     <PriceDisplay price={price} />
   </div>
  ); }

export default ProductPrice;
```

Explanation: The parent component now owns the single price state. It passes the current price and setPrice function to PriceInput through props. PriceInput changes the shared state, and PriceDisplay receives the same updated value. Because there is only one source of truth, both components remain synchronized.

6. Custom Hook useLocalStorage

A custom hook allows reusable React logic to be placed in one function. The useLocalStorage hook below behaves similarly to useState, but it also reads the initial value from localStorage and saves new values there whenever the state changes.

Answer / Corrected Code:

```
import { useEffect, useState } from "react";

function useLocalStorage(key, initialValue) {   const [value,
setValue] = useState(() => {     const savedValue =
localStorage.getItem(key);

   return savedValue !== null      ?
JSON.parse(savedValue)
     : initialValue;   });

  useEffect(() => {
   localStorage.setItem(key, JSON.stringify(value));   }, [key, value]);

  return [value, setValue]; }

export default useLocalStorage;
```

Explanation: First, the hook checks whether a value already exists in localStorage. If it exists, JSON.parse() converts it back into a JavaScript value; otherwise, initialValue is used. Whenever the value changes, useEffect() saves the new value using localStorage.setItem(). The hook returns [value, setValue], so it can be used almost like useState.

7. Fixing the React Key Warning

When React renders a list using map(), every list item should have a unique key. The key helps React identify which item has changed, been added, or been removed. The original code does not provide a key, so React displays a warning and list updates can be less efficient.

Answer / Corrected Code:

```
{items.map(item => (
  <li key={item.id}>
   {item.name}
  </li>
))}
```

Explanation: The key should be a stable and unique value for each item. If every item has a unique id, item.id is a good choice. React uses these keys to efficiently compare the previous list with the new list. Avoid using the array index as a key when list items can be reordered, inserted, or removed.

8. Reusable Button Component with a variant Prop

A reusable component can accept props to change its appearance or behavior. Here, the Button component accepts a variant prop with values such as primary or danger. The value is then used to select the appropriate CSS class.

Answer / Corrected Code:

```
function Button({ variant = "primary", children }) {   const className =     variant ===
"danger" ? "btn btn-danger" : "btn btn-primary";

  return (    <button className={className}
>
    {children}
   </button>
  ); }
```

```
export default Button; /*

Example usage:

<Button variant="primary">Save</Button>
<Button variant="danger">Delete</Button>

*/

/* CSS .btn {
  padding: 10px 18px;
border: none;   border-
radius: 5px;   cursor: pointer
; }

.btn-primary
{   background: blue;
color: white; }

.btn-danger
{   background: red;
color: white; }
*/
```

Explanation: If variant is "danger", the button receives the btn-danger class. Otherwise, it receives btn-primary. The children prop allows the same component to display different button text, such as Save, Delete, Update, or Submit.

9. Preventing Unnecessary Child Re-renders with React.memo

Normally, when a parent component re-renders, its child components are also rendered again. In this example, changing tick causes ParentDashboard to re-render. ChildCard receives the same title prop, but without memoization React can still render ChildCard again. React.memo can prevent the child from re-rendering when its props have not changed.

Answer / Corrected Code:

```
import React, { useState } from "react";

const ChildCard = React.memo(function ChildCard({ title })
{   console.log("ChildCard rendered");

  return <h3>{title}</h3>; });

function ParentDashboard() {   const [tick,
setTick] = useState(0);

  return (     <div>
     <button onClick={() => setTick(tick + 1)}>
       Refresh
     </button>

     <p>Refresh count: {tick}</p>

     <ChildCard title="Static Title" />
   </div>
 ); }

export default ParentDashboard;
```

Explanation: ParentDashboard changes its state whenever Refresh is clicked, so the parent renders again. React.memo remembers the previous props of ChildCard. Because title remains the same, React can skip rendering ChildCard again. This can improve performance when a child component is expensive to render.

10. React Router v6 with Three Routes

React Router is used to create navigation between different pages or views in a React application without

requiring a complete browser page reload. The required routes are /, /products, and /products/:id. Link components provide navigation in the navbar.

Answer / Corrected Code:

```
import {   BrowserRouter,
  Routes,
  Route,
  Link } from "react-router-dom";

function Home() {   return
<h2>Home Page</h2>; }

function Products()
{   return (     <div>
     <h2>Products Page</h2>      <Link to="/products/1">Open
Product 1</Link>
   </div>
  );
}
function ProductDetails() {   return <h2>Product
Details Page</h2>; }

function App() {   return
(    <BrowserRouter>
     <nav>
       <Link to="/">Home</Link>{" | "}
       <Link to="/products">Products</Link>{" | "}
       <Link to="/products/1">Product 1</Link>
     </nav>

     <Routes>
       <Route path="/" element={<Home />} />
       <Route path="/products" element={<Products />} />        <Route
path="/products/:id"          element={<ProductDetails />}        />
     </Routes>
   </BrowserRouter>
  ); }

export default App;
```

Explanation: BrowserRouter provides routing support. Routes contains the route definitions, and each Route connects a URL path with a React component. The :id part is a dynamic route parameter, so URLs such as /products/1 and /products/25 can represent different product IDs. Link changes the route without performing a normal full-page reload.

11. Refactoring Form State into One Object

Having a separate useState hook for every form field can make a larger form repetitive. A better approach is to keep all fields inside one state object and use one shared onChange handler. The name attribute of each input tells the handler which property should be updated.

Answer / Corrected Code:

```
import React, { useState } from "react";

function ContactForm() {   const [form,
setForm] = useState({     name: "",     email:
"",     phone: ""   });

  const handleChange = (event) => {     const { name,
value } = event.target;

    setForm(prevForm => ({       ...prevForm,
      [name]: value
    }));   };

  const handleSubmit = (event) =>
{    event.preventDefault();    console.log(form
```

```
);  };

 return (    <form onSubmit={handleSubmit}
>     <input       name="name"
value={form.name}
onChange={handleChange}
placeholder="Name"      />

    <input       name="email"
value={form.email}
onChange={handleChange}
placeholder="Email"      />

    <input       name="phone"
value={form.phone}
onChange={handleChange}
placeholder="Phone"      />

    <button type="submit">Submit</button>
  </form>
 ); }

export default ContactForm;
```